Profile-Guided-Optimisation of LatticeQCD contraction on CPU and GPU

January 2026

1 Introduction

This project focuses on targeted performance improvement of the Lattice QCD computation of the hadronic light-by-light (HLbL) contribution to the anomalous magnetic moment of the muon. Lattice QCD computations are of statistical nature. Calculations are repeated until the desired statistical significance is reached. Thus, improving its performance is critical for accelerating scientific research and saving computational resources. Profilers are employed to identify bottlenecks in both CPU and GPU implementations, which are addressed with a combination of code modernisation, better memory handling, maximised parallelism, and low-level improvements.

Firstly, the computationally intense components of the code are identified. Measu performance is important as makes the bottlenecks of the program apparent so that optimization effort can be focused on the most critical regions, leading to cost-effective performance improvement.

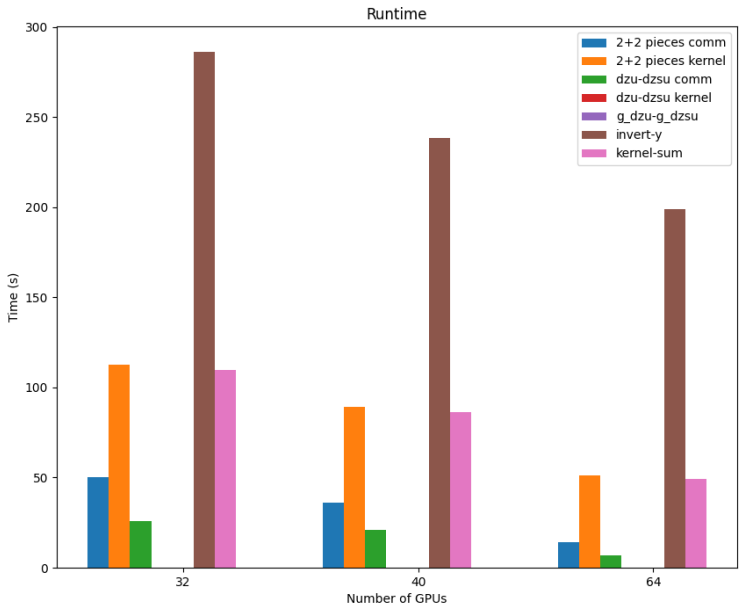

Figure 1: An overview of the time spent in different parts of the HLbL code with increasing number of GPUs. The red dzu-dzsu kernel and purple g_dzu-g_dzsu bars are invisible because they complete within a millisecond. The performance analysis of HLbL code on grid size "cC", i.e. $160 \times 80 \times 80 \times 80$.

Timers are added to the different components of the HLbL code. The code was run on cluster daint (4 gh200 per node) from CSCS with different number of nodes. The time spent on different components are

shown in Figure 1. Overall, invert-y is the most demanding part of the code. However, inverty-y involves calling a already state-of-the-art solver from the external library QUDA. Therfore, we focus energy on the second most demanding part which would be 2+2 pieces computation and communication.

The 2+2 contraction is first decomposed into smaller modules for easier profiling and optimisation. The CPU optimisations included loop re-arrangement to improve data locality, the replacement of multi-level nested memory allocations with contiguous 1D pointers, and the resolution of thread-synchronization bottlenecks in parallel regions. L1 cache miss rates see multi-factor reductions and parallelised regions show near perfect strong scaling across the modules. On GPUs, we rearranged data access pattern, improved parallel work distribution, and identified latency issues. Finally, kernels are tied together with asynchronous streaming to overlap workload at small problem size and hide communication and data copy latencies. The optimized code achieved an overall 30% runtime reduction in production environments (GPU) on CSCS Daint. This project highlights how systematic profiling and targeted optimizations can yield significant resource savings in computationally intensive legacy code.

2 2+2 contraction

From the computational stand point, the 2+2 contraction code takes an input forward propagator and an array of point pairs and returns three output vectors P1, P2 and P3 which do not scale with volume.

Figure 2 illustrates the dependencies of the 2+2 contractions. The forward propagator is a complex object of dimension Volume $\times (3 \times 4)^2$. Volume is the number of points on the lattice, 3 is the dimension of the colour index, and 4 is the dimension of the Lorentz/spinor index. It undergoes the first stage of contraction into the intermediate Pi, which is a real object of dimension Volume $\times 4 \times 4$. From here, P1 can already computed. As for P2 and P3, they both depend on a set of point pairs that is also given as input. The coordinates of the point pairs are processed and passed to an KQED kernel based on an externel library that returns a $6 \times 4 \times 4 \times 4$ matrix. Finally, this matrix is contracted with Pi to give P2 and P3.

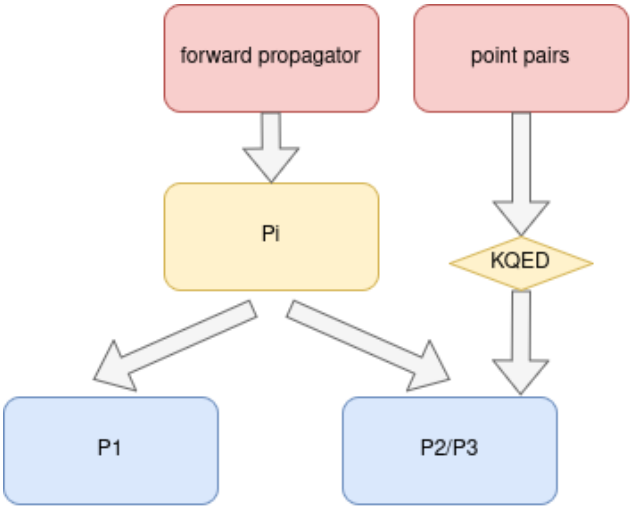

Figure 2: A diagram showing the dependency of different parts of the 2+2 contraction

3 CPU opitmisation

The original CPU code is a single C++ function of over 300 lines. For better readability and easier profiling, this function is first broken down into 3 parts: compute_pi, compute_p1, and compute_p23. Each part was profiled and optimised separately.

3.1 compute_pi

Upon close inspection, it is clear that although compute_pi contraction is inherently memory-bound, it is embarrassingly parallel and highly-optimisable. The main part of the computation is a triple loop over 3 indices, the volume and two dimension-4 Lorentz indices. If we see the forward propagator as a lattice with a 12×12 matrix at every point, the algorithm only operates on the local matrix at every point, in other words, the computation at every point is completely independent. The most effective way of parallelisation would be over the volume dimension, such that zero communication is needed and all data accesses are local. However, the original implementation loops over the two local indices before volume. Despite parallelising over the volume dimension, it did not take full advantage of the parallelism.

A simple re-arrangement of loop order by putting the volume loop first provides massive improvements in temporal and spatial data locality for this memory-bound algorithm. The 12×12 matrix fit very well in cache. In the ideal case where all local processing fit in cache, the memory footprint could reduce by as much as 4×4. Additional small improvements came from explicit caching of the 4×4 matrix at each lattice point.

On a test run with 16 threads on a laptop with AMD Ryzen 9 7940HS chip, `perf stat` reported significantly lower L1 cache misses from 7.2% to 0.52%, higher instruction counts per cycle from 2.01 to 2.29, and 40% improved runtime.

Original version:

```
Performance counter stats for 'system wide':

          0.00 msec cpu-clock                 #0.000 CPUs utilized
       107,623      context-switches
         2,167      cpu-migrations
        75,861      page-faults
643,231,587,856      instructions              #2.01  insn per cycle
                                               #0.10  stalled cycles per insn   (71.43%)
319,920,997,578      cycles                                                      (71.43%)
63,135,983,970       stalled-cycles-frontend #19.73% frontend cycles idle        (71.43%)
17,238,764,328       branches                                                    (71.43%)
   553,805,305       branch-misses           #3.21% of all branches              (71.43%)
292,778,242,944      L1-dcache-loads                                             (71.43%)
21,081,766,719       L1-dcache-load-misses   #7.20% of all L1-dcache accesses    (71.43%)

  51.269802002 seconds time elapsed
```

Optimised version:

```
Performance counter stats for 'system wide':

          0.00 msec cpu-clock                 #0.000 CPUs utilized
        55,180      context-switches
           724      cpu-migrations
        47,743      page-faults
434,126,153,062      instructions              #2.29  insn per cycle
                                               #0.09  stalled cycles per insn   (71.43%)
189,892,221,114      cycles                                                      (71.43%)
39,491,806,621       stalled-cycles-frontend #20.80% frontend cycles idle        (71.43%)
15,540,362,297       branches                                                    (71.42%)
   329,847,465       branch-misses           #2.12% of all branches              (71.42%)
133,697,461,145      L1-dcache-loads                                             (71.44%)
   693,719,997       L1-dcache-load-misses   #0.52% of all L1-dcache accesses    (71.44%)

  29.718351883 seconds time elapsed
```

3.2 compute_p1

Similar to compute_pi, compute_p1 also has nested loop that did not take into account of data locality. Given the contraction is memory-bound, we can applying data-oriented optimisation to maximise data reuse. After careful examination of the data access pattern and loop order adjustments, the L1 cache miss rate is down four-fold.

A 4-level nested dynamically allocated butter was replaced by a 1D pointer with a single allocation. The original allocation stays close to the physics understanding of the object, a 4D object.

```
    double **** local_P1 = (double ****)malloc(sizeof(double ***) *4);
  for (int i=0; i<4; i++){
    local_P1[i] = (double ***)malloc(sizeof(double **) *4);
    for (int j=0; j<4; j++){
      local_P1[i][j] = (double **)malloc(sizeof(double *) *4);
      for (int k=0; k<4; k++){
        local_P1[i][j][k] = (double *)calloc(Lmax, sizeof(double));
      }
    }
  }
```

However, it is a known problem that allocation like this means that the actual memory of local_P1 is not guaranteed to be continuous. To simplify the code and guarantee continuity, this is replaced by a single allocation of the full-length of local_P1. When profiled, the single allocation version performed better in branch missed and frontend stall, possibly due to the simplified logic.

Below is the metrics measured by `perf` for both the original code and optimised code. The runtime after optimisation reduced by around 40%, similar to compute_pi.

Original version:

```
    Performance counter stats for 'system wide':

                 0      cpu-clock                 #0.000 CPUs utilized
           134,041      context-switches
             3,561      cpu-migrations
           117,344      page-faults
   654,667,589,500      instructions              #2.04  insn per cycle
                                                  #0.10  stalled cycles per insn (71.43%)
   320,302,832,460      cycles
(71.43%)
    64,137,438,954      stalled-cycles-frontend #20.02% frontend cycles idle  (71.43%)
    19,401,537,280      branches                                              (71.43%)
       592,928,648      branch-misses            #3.06% of all branches       (71.43%)
   297,980,630,791      L1-dcache-loads                                       (71.43%)
    20,218,868,660      L1-dcache-load-misses #6.79% of all L1-dcache accesses(71.43%)

      49.837460664 seconds time elapsed
```

Optimised version:

```
    Performance counter stats for 'system wide':

                 0      cpu-clock                 #0.000 CPUs utilized
           212,584      context-switches
             5,815      cpu-migrations
           149,425      page-faults
   478,252,367,450      instructions              #2.17  insn per cycle
                                                  #0.10  stalled cycles per insn(71.43%)
   220,334,119,511      cycles                                                (71.43%)
```

```
46,275,352,263      stalled-cycles-frontend #21.00% frontend cycles idle (71.43%)
22,605,990,399      branches                                             (71.42%)
   650,030,868      branch-misses           #2.88% of all branches       (71.43%)
150,248,539,766     L1-dcache-loads                                      (71.43%)
 1,380,508,295      L1-dcache-load-misses #0.92% of all L1-dcache accesses(71.43%)

      30.048695746 seconds time elapsed
```

3.3 compute_p23

P2 can be divided into P2_0 and P2_1. P2_0, P2_1 and P3 calculations are very similar. They all take Pi and an array of point pairs as input and output an $3 \times 4 \times 4 \times 4$ each, where the 3 comes from 3 types of QED kernels, and the others are three Lorentz indices. The implementation loops over the lattice volume, the point pairs and the 3 QED kernel types, computes the QED contribution based on the coordinates and the kernel type, contracts the QED contribution with Pi and finally accumulates to output.

The original code is not thread-parallelised due to the complex dependencies. The integration range is the entire lattice volume, and a threaded reduction is difficult. However, technically the computation is parallel in point pairs and kernel type and we can potentially parallelise over the two.

We decided on the simplest option to move point pairs to the outermost loop. Given the number of point pairs in the order of the number of threads, this is a very good options for parallelisation. This resulted in nearly over two times time reduction.

However, at this point it was discovered that multi-thread scaling was terrible. In fact, increasing thread count barely reduces runtime. High frontend stall and increased L1 misses was also observed. Firstly, we considered possibility of false sharing, meaning multiple threads are attempting to access the same region of data simultaneously and are unable to, resulting in long stalls and a serialised program. But it was ruled out as padding all allocations did not lead to improvement. Then, we measured L1 instruction and data cache misses using **perf record -e -L1-dcache-misses,L1-icache-misses** (exact command may be machine dependent) to gain more information on the frontend stall.

Abnormal L1 cache miss event count, limited by `__random`:

```
Samples: 125K of event 'L1-dcache-misses', Event count (approx.): 1883988950

Overhead  Command  Shared Object          Symbol

  21.51%  main     libc.so.6              [.] __random

   9.70%  main     [kernel.kallsyms]      [k] __futex_hash

   8.54%  main     [kernel.kallsyms]      [k] do_syscall_64

   8.16%  main     [kernel.kallsyms]      [k] futex_hash

   6.94%  main     main                   [.] compute_p23(double*, double (*) [79][4][4][4],

Samples: 24K of event 'L1-icache-misses', Event count (approx.): 1288524

Overhead  Command  Shared Object              Symbol

  11.98%  main     main                       [.] compute_p23(double*, double (*) [79][4][4

   7.47%  main     libc.so.6                  [.] __random

   7.44%  main     [kernel.kallsyms]          [k] perf_ctx_disable
```

```
6.48%  main  [kernel.kallsyms]  [k] native_write_msr

5.71%  main  [kernel.kallsyms]  [k] __perf_event_task_sched_out
```

The abnormally high 21.51% dcache and 7.47% icache misses from `__random` was especially alarming. Finally, it is identified that due to a careless random initialization call in an OMP parallel region, all threads had to share the same seedrandom number generator, resulting and stalling. Removing the parallel region effectively addresses the problem. L1 dcache misses immediately dropped by half to $65k$, and icache misses dropped four times to $6k$.

To cross check, valgrind and kcachegrind were also used to examine the performance of compute_p23. The callgraph indicates most of the work is done doing compute_p23 with minimal OMP overhead. This could mean excellent scaling, which was indeed observed.

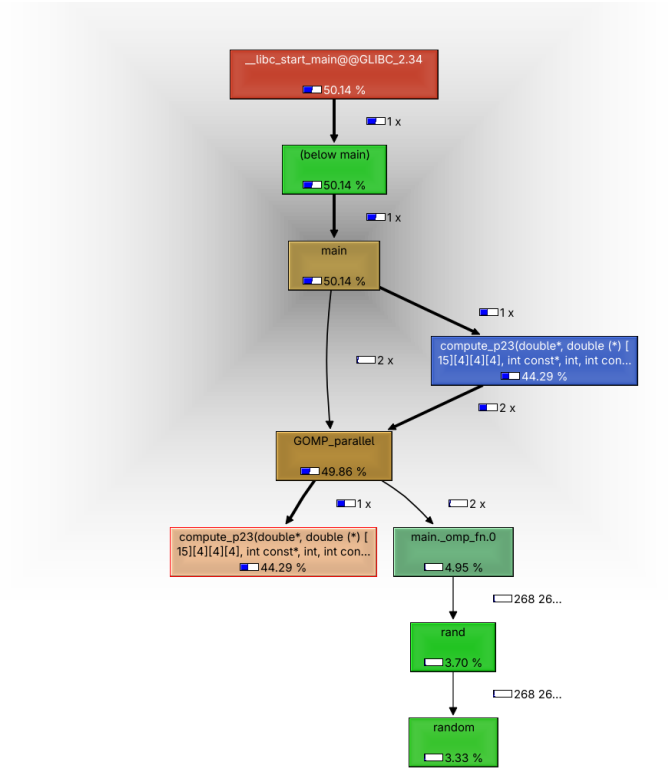

Figure 3: Callgraph with relative percentage of instructions of one of the threads in a 2-OMP-thread run of compute_p23. Nearly all instructions used for compute_p23, minimal overhead.

KCachegrind also has the option of showing instruction fetch line by line, which helped identify a region of high register pressure. Unrolling a search table and explicit caching by using static local variable to store reusable data provided further speed up. The overall serial speed up is 56%. Strong scaling matches expectation.

Original version (serial):

```
 Performance counter stats for 'system wide':

             0      cpu-clock                 #0.000 CPUs utilized
       128,919      context-switches
         1,958      cpu-migrations
        62,355      page-faults
723,494,192,030      instructions              #2.51  insn per cycle
```

```
                                          #0.05  stalled cycles per insn   (71.43%)
288,557,990,883      cycles                                                (71.43%)
37,881,145,391       stalled-cycles-frontend #13.13% frontend cycles idle   (71.43%)
47,879,513,458       branches                                              (71.42%)
   421,382,752       branch-misses           #0.88% of all branches        (71.43%)
347,978,333,990      L1-dcache-loads                                       (71.44%)
   795,641,407       L1-dcache-load-misses   #0.23% of all L1-dcache accesses (71.42%)

  50.105110039 seconds time elapsed
```

Scaling after optimisation (parallelised):

```
1 thread:
Performance counter stats for 'system wide':

             0      cpu-clock               #0.000 CPUs utilized
       413,484      context-switches
        14,788      cpu-migrations
       255,875      page-faults
557,714,976,949      instructions            #2.90  insn per cycle
                                             #0.06  stalled cycles per insn    (71.43%)
192,032,075,918      cycles                                                    (71.43%)
35,385,445,488       stalled-cycles-frontend #18.43% frontend cycles idle      (71.43%)
59,236,308,278       branches                                                  (71.43%)
   483,008,520       branch-misses           #0.82% of all branches            (71.43%)
201,560,347,589      L1-dcache-loads                                           (71.43%)
 2,634,033,059       L1-dcache-load-misses   #1.31% of all L1-dcache accesses  (71.43%)

  21.944105772 seconds time elapsed
2 threads:
Performance counter stats for 'system wide':

             0      cpu-clock               #0.000 CPUs utilized
       253,390      context-switches
        12,500      cpu-migrations
       164,192      page-faults
523,389,039,699      instructions            #3.34  insn per cycle
                                             #0.04  stalled cycles per insn    (71.43%)
156,699,510,631      cycles                                                    (71.43%)
23,509,958,267       stalled-cycles-frontend #15.00% frontend cycles idle      (71.43%)
53,428,165,779       branches                                                  (71.43%)
   303,593,469       branch-misses           #0.57% of all branches            (71.43%)
189,379,065,274      L1-dcache-loads                                           (71.43%)
 1,713,630,863       L1-dcache-load-misses   #0.90% of all L1-dcache accesses  (71.43%)

  13.104141048 seconds time elapsed

4 threads:
Performance counter stats for 'system wide':

             0      cpu-clock               #0.000 CPUs utilized
       176,110      context-switches
        11,571      cpu-migrations
       118,537      page-faults
506,997,589,733      instructions            #3.48  insn per cycle
```

```
                                                  #0.04   stalled cycles per insn     (71.42%)
145,585,905,120      cycles                                                          (71.41%)
19,657,780,079       stalled-cycles-frontend     #13.50% frontend cycles idle       (71.42%)
51,239,528,096       branches                                                        (71.42%)
   235,349,861       branch-misses               #0.46% of all branches             (71.44%)
183,494,775,102      L1-dcache-loads                                                 (71.46%)
 1,332,969,531       L1-dcache-load-misses       #0.73% of all L1-dcache accesses   (71.44%)

   8.500924173 seconds time elapsed
```

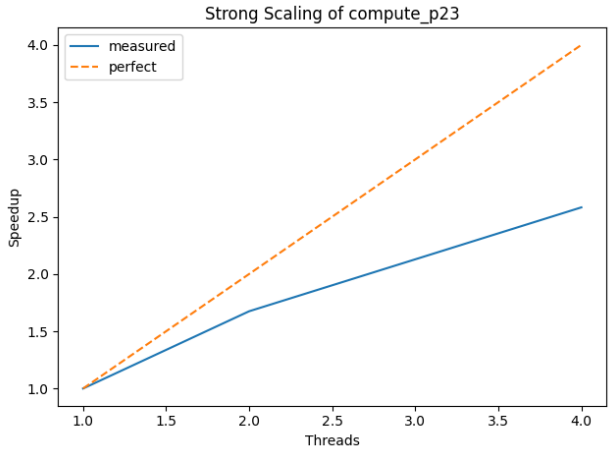

Figure 4: Strong scaling of compute_p23

4 GPU optimisation

The original GPU code is a long single function call. But unlike the CPU code, the GPU adaptation has already gone through one round of optimisation. For example, to possibly reduce memory footprint, the author opted for local operation only, and implemented one large loop over volume that computes Pi locally and accumulates to respective parts of P1, P2_0, P2_1, and P_3. However, a running issue is that the code is still very much structured in a serial way, inherited from the CPU code, which is highlighted by profiles this issue: register pressure, store spills, under-utilised parallelism etc. Given the cache heavy operation at every point and complex dependencies, the 2+2 contraction is not the most suitable for GPU architectures, nor is it straightforward to adapt.

This round of optimisation restructred the code completely. The single big loop over volume is removed, the intermediate array Pi is restored. It is decided that Pi scales linearly with volume and do not incur so much memory footprint. Instead, the unified memory of the target architecture (GraceHopper) can be used to cut down memory demand on the GPU.

4.1 kernel_pi

Similar to the CPU optimisation of compute_pi, each GPU thread is assigned to tackle the computation at one lattice point. The process is embarrassingly parallel. The naive implementation however, suffers from register pressure. By setting compiler flag `--ptxas-options=-v` and profiling using `ncu`, high register requirement spill stores were observed. Applying unrolling to look-up table and removing loops by specifying block dimension relieves the problem. In the case of kernel_pi, we need to loop over x, which is size Volume, and μ, ν, ia which are of sizes 4, 4, and 12. By choosing to parallelise x over GPU grid, and setting GPU block size to $(4, 4, 12)$, or 192 threads, to match the μ, ν, exactly (i.e. `int mu = threadIdx.x;`), we not

only save 3 "for" loops, but we can also preload the information at point x into shared memory to be used by all 192 threads, reducing register requirement per thread.

The final version of kernel_pi has 0 spills and uses 127 register, meaning given the 255 register per SM limit of GraceHopper, each SM can potentially support 2 simultaneous blocks.

```
ptxas info    : Compiling entry function '_Z9kernel_piPdS_ij' for 'sm_90'
ptxas info    : Function properties for _Z9kernel_piPdS_ij
    192 bytes stack frame, 0 bytes spill stores, 0 bytes spill loads
ptxas info    : Used 127 registers, used 1 barriers, 6144 bytes smem
```

In reality, 2 blocks per SM is not achieved because the program is memory bound as expected. The Nsight profile shows that kernel_pi ran at 71.70% memory bandwidth. Since kernel_pi is not the bottleneck of 2+2 computation, this performance is deemed satisfactory, we focus opitmisation effort on more demanding parts of the code.

```
kernel_pi(double *, double *, int, unsigned int) (132, 1, 1)x(4, 4, 12), Context 1, Stream
    Section: GPU Speed Of Light Throughput
```

Metric Name	Metric Unit	Metric Value
DRAM Frequency	Ghz	2.62
SM Frequency	Ghz	1.53
Elapsed Cycles	cycle	21,228,501
Memory Throughput	%	71.70
DRAM Throughput	%	0.01
Duration	ms	13.87
L1/TEX Cache Throughput	%	56.97
L2 Cache Throughput	%	71.70
SM Active Cycles	cycle	21,071,978.22
Compute (SM) Throughput	%	17.27

4.2 kernel_p1

In the CPU version P1 computation was kept serial because of the complexity. The algorithm loops over all location on the lattice, and at each point x loops over the 4 dimensions $x[0]$, $x[1]$, $x[2]$, and $x[3]$, compute some intermediate and accumulate it to P1[x[0]], P1[x[1]], P1[x[2]], and P1[x[3]]. This is a slightly unconventional kernel and the original GPU adaptation maintained the same CPU code but instead used the blocking `atomicAdd_system` to accumulate to the P1 in global memory. To make the algorithm less serial and make use of the GPU threads better, this round of optimisation inverted the algorithm by looping over the length of P1, and computing all points in the volume with a dimension that corresponds to that P1 position. This way, a lot more accumulation can be computed thread-local, and only one atomicAdd_system is needed per P1 location (40 total) as opposed to four per lattice point (4×40^4).

The resultant algorithm is sub-second and very fast. Further optimisation is unnecessary.

4.3 kernel_p20, kernel_p21, kernel_p3

The computation of P2 and P3 are the heaviest part of 2+2 computation on the GPU. A standard reduction with nitially implemented with `atomicAdd_system` was changed for a more optimised version with warp-level primitives. But the main issue was register usage. Since every thread needs to call function `KQED_LX` from an externel library that returns a $6 \times 4 \times 4 \times 4$ object that translates to over 700 registers, way more than the 255 register a whole SM can provide, the program uses the maximum 255 registers, cannot support multiple concurrent blocks, and is heavily latency-bound. P2 and P3 computation is split into 3 smaller kernels to alleviate the register demand, but the latency problem persists. We also attempted to limit the number of registers per thread manually to below 128 by applying launch bound, but it did not return good improvements. In the end, the 3 kernels run at around 20% the GPU capacity.

Unless the externel KQED library is rewritten in a GPU friendly way, the latency issue from too-large register requirement will likely not be resolved. This is an unfortunate side effect of CPU libraries being translated to GPU without taking into account the different limitations of the GPU.

```
kernel_p20(double *, double *, int, const int *, const int *, const double *, QED_kernel_te
    Section: GPU Speed Of Light Throughput
```

Metric Name	Metric Unit	Metric Value
DRAM Frequency	Ghz	2.62
SM Frequency	Ghz	1.53
Elapsed Cycles	cycle	21,884,043,860
Memory Throughput	%	20.49
DRAM Throughput	%	5.25
Duration	s	14.30
L1/TEX Cache Throughput	%	20.31
L2 Cache Throughput	%	20.49
SM Active Cycles	cycle	17,028,260,557.65
Compute (SM) Throughput	%	11.64

```
    OPT   This kernel exhibits low compute throughput and memory bandwidth utilization rela
          of this device. Achieved compute throughput and/or memory bandwidth below 60.0% o
          latency issues. Look at Scheduler Statistics and Warp State Statistics for poten

    Section: Launch Statistics
```

Metric Name	Metric Unit	Metric Value
Block Size		256
Cluster Scheduling Policy		PolicySpread
Cluster Size		0
Function Cache Configuration		CachePreferNone
Grid Size		264
Registers Per Thread	register/thread	255
Shared Memory Configuration Size	Kbyte	8.19
Driver Shared Memory Per Block	Kbyte/block	1.02
Dynamic Shared Memory Per Block	byte/block	0
Static Shared Memory Per Block	byte/block	256
# SMs	SM	132
Threads	thread	67,584
Uses Green Context		0
Waves Per SM		2

4.4 Stream

CUDA has a stream feature that allows different small processes to run on the same GPU without interference. Since kernel_p1, kernel_p20, kernel_p21, and kernel_p3 are independent, they can be computed in four separate streams. This way, the device to host data copy and communication can also be broken down to small parts and be overlapped or hidden.

Streaming achieved positive effect when the P2/3 computation load is small, i.e. when number of pairs is small. The kernels overlap effectively, and data copy (the pink bits at the end of kernel) is hidden completely.

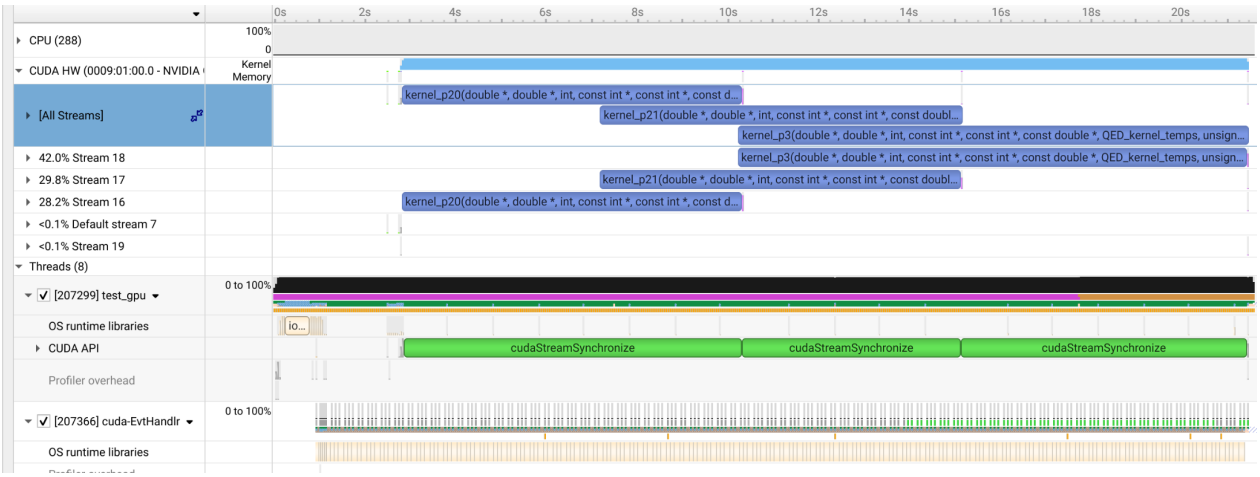

Figure 5: Visible overlap of kernels when the workload of individual kernels are small (44 pairs).

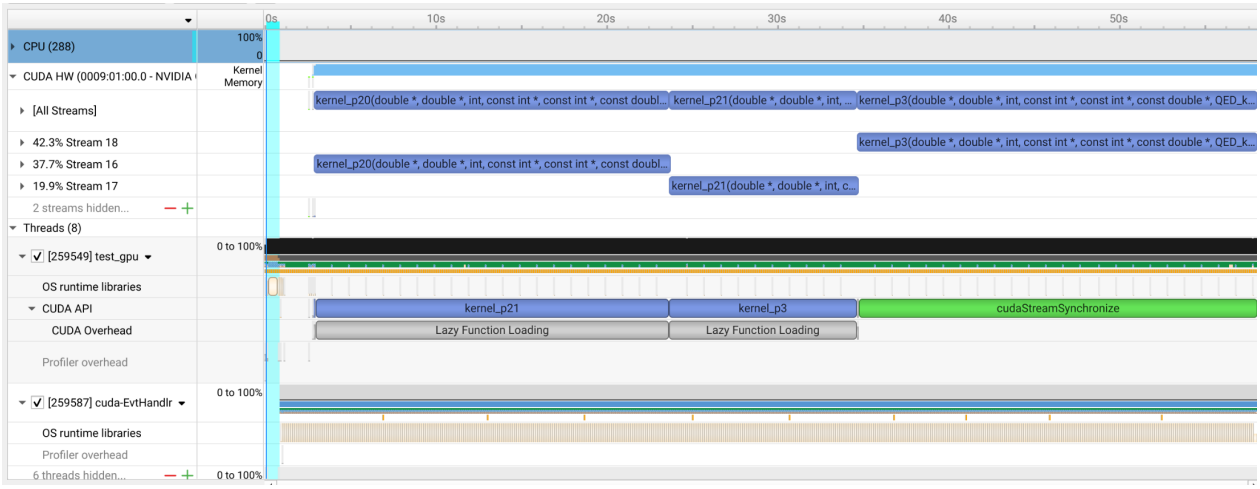

Figure 6: No overlap and nearly serial workflow when individual kernels can fill the entire GPU (132 pairs).

However, at larger work load, when the GPU can be saturated by a single kernel, the benefit of streaming is less pronounced and has the same performance as using the default stream. But overall, streaming is a nice addition that helps make best use of the GPU when individual kernels are under-utilising the GPU, saving energy and resources.

4.5 Performance in production code

The performance improvement is apparent when applied to production code. The original time is 195.85 seconds computation plus 33.83 seconds of communication, or 229.68 seconds total. The new optimised time with overlapping communication is 182.67 seconds total. Thanks to the hidden communication and kernel optimisation, the 2+2 computation sees around 30% runtime reduction overall.